Universidade Federal de São Carlos

Centro de Ciências Exatas e de Tecnologia
Departamento de Ciência da Computação

Trabalho de Aprendizado de Máquina 2

X-ray Lung Diseases Images

Alunos:

Camila Oliveira de Souza	RA: 800361
Davi Cerchiari Alves	RA: 813985
Ivan Capeli Navas	RA: 802286
Pedro Marchi Nunes	RA: 801053
Vitor Yuki Inumaru Ferreira	RA: 794041

Dezembro, 2025

Conteúdo

1. Introdução **3**
2. Fundamentação Teórica **3**
2.1. CNN 3
2.2. Bibliotecas Utilizadas 4
2.2.1. Frameworks de Aprendizado Profundo (Deep Learning) 4
2.2.2. Processamento Digital de Imagens e Extração de Características 4
2.2.3. Manipulação de Dados e Análise Estatística 5
2.2.4 Avaliação de Desempenho e Visualização 5
3. Metodologia **5**
3.1. Base de Dados 5
3.2. Pré-Processamento 7
3.3. Extração das Features e Tratamento das Imagens 7
3.4. Aplicação dos Algoritmos 9
3.4.1. CNN Simples 10
3.4.2 Autoencoder + CNN Simples 11
3.4.3 Modelo Baseado em ResNet 15
4. Visualização e Comparação dos Resultados **17**
5. Conclusão **19**
6. Referências **20**

1. Introdução

Este trabalho tem como objetivo o desenvolvimento e avaliação de modelos de aprendizado de máquina voltados para a classificação automática de patologias pulmonares e alterações torácicas a partir de imagens de Raio-X. O diagnóstico por imagem desempenha um papel crucial na medicina moderna, e a aplicação de técnicas de Visão Computacional visa auxiliar profissionais de saúde, oferecendo suporte à decisão clínica com maior agilidade e precisão.

Para este estudo, foi utilizado o conjunto de dados "X-Ray Lung Diseases Images" (link), disponível na plataforma Kaggle. A base de dados é composta por imagens de radiografias de tórax categorizadas em 9 classes distintas, abrangendo desde condições normais até patologias complexas, como alterações de mediastino (incluindo pericardite, malformações arteriovenosas e linfonodomegalias), além de outras afecções pulmonares.

O foco principal do projeto reside no pré-processamento destas imagens, na extração de características relevantes e no treinamento de classificadores capazes de distinguir entre as diferentes etiologias presentes no dataset, avaliando-se métricas de desempenho para validar a eficácia da abordagem proposta. Através desse link é possível acessar o código do colab.

2. Fundamentação Teórica

2.1. CNN

Para a tarefa de classificação das imagens de Raio-X, optou-se pela utilização de Redes Neurais Convolucionais (CNNs). Esta classe de algoritmos de Deep Learning representa o estado da arte em problemas de Visão Computacional, sendo escolhida devido à sua capacidade de aprender automaticamente hierarquias espaciais de características (features) diretamente a partir dos dados brutos, dispensando a extração manual de atributos complexos.

A arquitetura do modelo fundamenta-se no processamento sequencial da informação visual através de dois blocos principais:

- **Extração de Características**: Camadas de convolução aplicam filtros treináveis sobre a imagem para detectar padrões locais, como texturas de tecido pulmonar, bordas de órgãos e opacidades características de patologias (ex: linfonodomegalias). Frequentemente intercaladas com camadas de pooling (subamostragem), que reduzem a dimensionalidade do mapa de características e conferem invariância a pequenas translações e distorções.

- **Classificação**: As características de alto nível extraídas são vetorizadas (flatten) e introduzidas em camadas densas (fully connected). A camada final utiliza uma função de ativação (como Softmax) para calcular a distribuição de probabilidade entre as 9 classes de patologias presentes no dataset.

2.2. Bibliotecas Utilizadas

Neste trabalho utilizamos diversas bibliotecas de Python, cada uma com um propósito específico no fluxo de processamento e modelagem de dados:

2.2.1. Frameworks de Aprendizado Profundo (Deep Learning)

A construção e o treinamento das redes neurais constituíram o núcleo deste projeto, utilizando-se de dois dos principais frameworks:

- **PyTorch (torch):** Foi a biblioteca principal utilizada para a construção da arquitetura da CNN. Módulos específicos como torch.nn (camadas convolucionais Conv2d, lineares Linear, ativações ReLU, Sigmoid) e torch.optim (otimizadores Adam e SGD) permitiram a definição flexível do grafo computacional e o ajuste fino dos pesos da rede.
- **TensorFlow / Keras:** Utilizado especificamente para a importação e manipulação da arquitetura VGG16 pré-treinada e suas funções de pré-processamento (preprocess_input), facilitando a aplicação de técnicas de Transfer Learning.

2.2.2. Processamento Digital de Imagens e Extração de Características

Antes de serem submetidas aos modelos, as radiografias passaram por etapas de tratamento e análise de textura utilizando:

- **Torchvision (torchvision.transforms):** Empregado para o pipeline de transformações das imagens (Data Augmentation), incluindo redimensionamento, normalização e conversão para tensores.
- **Scikit-Image (skimage):** Fundamental para a extração de descritores de textura "clássicos" e análise de imagem. Foram utilizadas funções para cálculo da Matriz de Coocorrência (GLCM - graycomatrix), Padrões Binários Locais (local_binary_pattern) e detecção de bordas (canny), visando enriquecer a análise dos dados.

- **PIL (Python Imaging Library):** Utilizada para manipulação básica de arquivos de imagem e operações geométricas simples.

2.2.3. Manipulação de Dados e Análise Estatística

Para a estruturação dos dados e cálculos estatísticos auxiliares:

- **Pandas e NumPy:** O pandas foi utilizado para o gerenciamento de metadados e estruturação dos datasets em DataFrames, enquanto o numpy forneceu a base para todas as operações matriciais e vetoriais de alta performance necessárias para o processamento numérico das imagens.
- **SciPy (scipy.stats):** Utilizada para o cálculo de momentos estatísticos das imagens, como assimetria (skew) e curtose (kurtosis), que serviram como features adicionais na análise exploratória.

2.2.4 Avaliação de Desempenho e Visualização

A validação dos resultados e a interpretabilidade do modelo foram conduzidas com:

- **Scikit-Learn (sklearn):** Essencial para a divisão estratificada dos dados (train_test_split) e para o cálculo das métricas finais de avaliação, incluindo Acurácia, Matriz de Confusão e Relatório de Classificação (Precision, Recall e F1-Score).
- **Matplotlib e Seaborn:** Bibliotecas responsáveis pela geração dos gráficos de desempenho (curvas de perda/acurácia) e pela visualização da matriz de confusão (sns.heatmap), permitindo uma análise visual clara dos erros e acertos do modelo.

3. Metodologia

3.1. Base de Dados

A base utilizada consiste em um conjunto de 6743 imagens de radiografias de tórax (X-ray) classificadas em 9 categorias distintas, de acordo com a característica mais relevante observada em cada imagem. As classes são:

1. Normal;
2. Processos inflamatórios (pneumonia);

3. Maior densidade (derrame pleural, consolidação atelectásica, hidrotórax, empiema);
4. Menor densidade (pneumotórax, pneumomediastino, pneumoperitônio);
5. Doenças pulmonares obstrutivas (enfisema, broncopneumonia, bronquiectasias, embolia);
6. Doenças degenerativas/infecciosas (tuberculose, sarcoidose, proteinose, fibrose);
7. Lesões encapsuladas (abscessos, nódulos, cistos, massas tumorais, metástases);
8. Alterações de mediastino (pericardite, malformações arteriovenosas, linfonodomegalias);
9. Alterações da caixa torácica (atelectasia, malformações, agenesia, hipoplasia).

Trata-se de uma base que contém apenas imagens sem qualquer informação sobre pacientes, previamente analisadas e classificadas por radiologistas e disponibilizadas exclusivamente para fins de pesquisa. Na figura 1, pode-se observar algumas amostras das imagens:

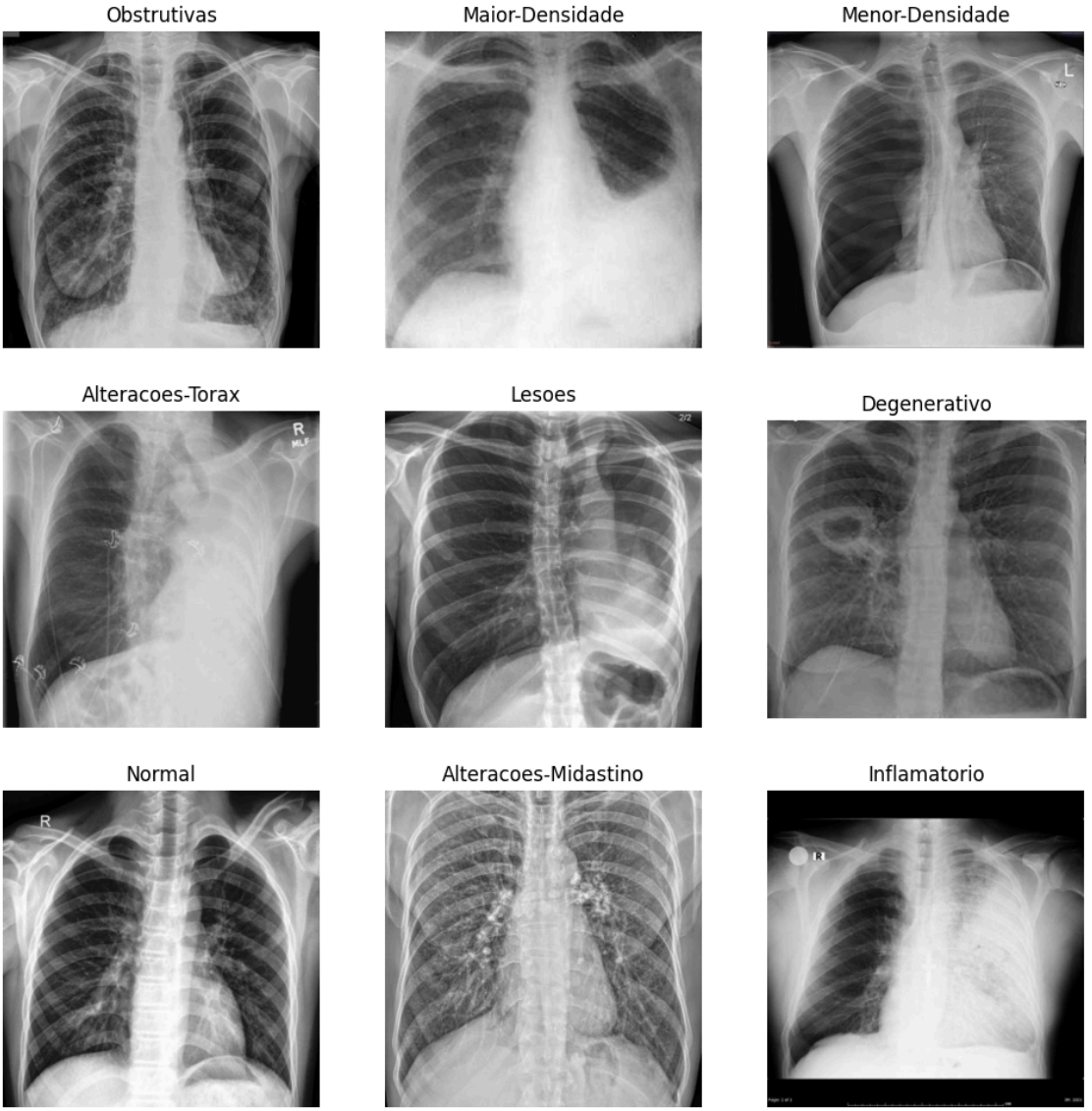

Figura 1

3.2. Pré-Processamento

O pré-processamento foi planejado com base na análise inicial da base, incluindo a verificação dos histogramas de intensidade e da distribuição das classes. Como as imagens apresentam variações de tamanho, contraste e iluminância, tornou-se necessário aplicar uma padronização consistente antes do treinamento dos modelos.

As etapas adotadas foram:

- **Conversão de cor**: As imagens foram carregadas em RGB para padronização e compatibilidade com modelos que esperam 3 canais. Também foi analisada a versão em escala de cinza (L), permitindo observar a distribuição de intensidades por meio do histograma.
- **Redimensionamento**: Todas as imagens foram uniformizadas para 112×112 pixels, garantindo entrada fixa para a rede neural.
- **Normalização**: Cada pixel foi escalado para o intervalo [0, 1], dividindo os valores por 255. Isso reduz variações indesejadas de intensidade e acelera o processo de treinamento.
- **Cache dos dados pré-processados**: Para evitar processamento repetido, as imagens tratadas foram salvas em um arquivo .pkl e reutilizadas nas execuções seguintes.

A função preprocess_image() centraliza esse pipeline, enquanto build_preprocessed_df() gera um DataFrame contendo: caminho da imagem, classe correspondente e o array da imagem pré-processada. A verificação final confirma que todas as imagens passaram a ter shape uniforme (112, 112, 3).

3.3. Extração das Features e Tratamento das Imagens

Antes de alimentar o modelo, foi feita uma etapa de inspeção e análise das imagens, visando compreender características relevantes da base e avaliar possíveis desbalanceamentos. As etapas realizadas foram:

- **Histograma de intensidades**: A conversão temporária para tons de cinza permitiu visualizar a distribuição de níveis de intensidade de pixels. Esse diagnóstico auxilia na

identificação de contrastes baixos ou artefatos, orientando decisões de normalização e augmentação.

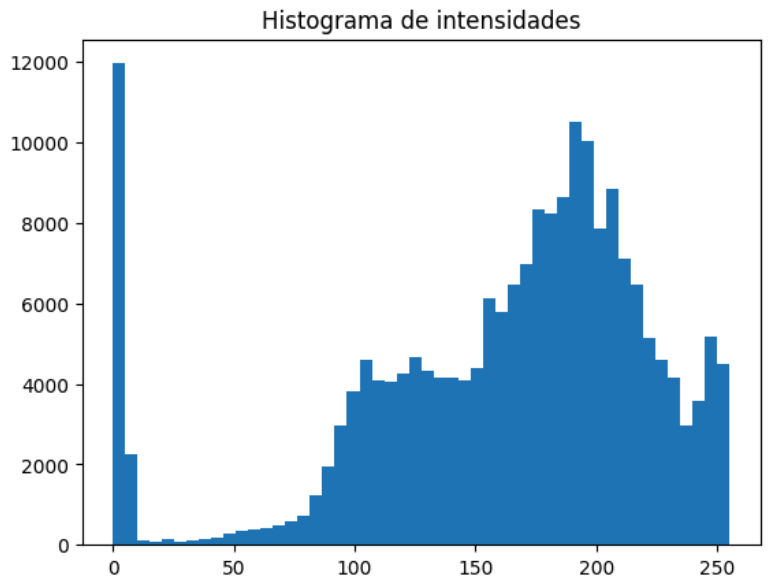

Figura 2

- **Análise da Distribuição das Classes**: Utilizando os.listdir e contagens por pasta, foi criado um gráfico de barras ilustrando o número de imagens em cada uma das 9 classes. Essa etapa revelou um desbalanceamento significativo, especialmente entre a classe Normal e categorias com menor representatividade, como Alterações de Mediastino.

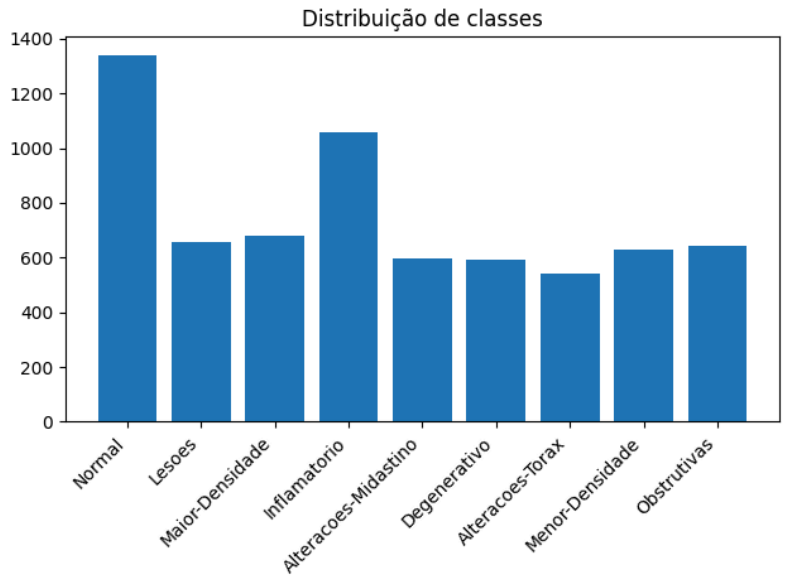

Figura 3

- **Cálculo de Pesos das Classes**: Para mitigar o desbalanceamento no treinamento, foram calculados o peso inversamente proporcional à frequência de cada classe. Os pesos finais

foram: [5.03, 6.36, 9.94, 10.72, 10.47, 11.35, 10.25, 11.31, 12.39]. Esses valores foram posteriormente usados como class weights no modelo, reduzindo o viés em direção às classes mais frequentes.

- **Visualização das amostras**: A função visualizar_aleatorio() foi criada para inspeção manual de imagens pré-processadas, exibindo amostras aleatórias com seus respectivos rótulos. Isso garante que o pipeline preserve a integridade e qualidade visual dos dados.

Figura 4

Essa etapa de extração e análise de features não utiliza features manuais (como HOG ou LBP), pois o projeto emprega redes neurais convolucionais que aprendem automaticamente representações internas. Assim, a fase de “feature extraction” consiste na inspeção estatística e visual dos dados, além da preparação adequada para que o modelo possa aprender de forma eficaz.

3.4. Aplicação dos Algoritmos

Para avaliar o desempenho em classificação das nove categorias da base de radiografias de tórax, foram implementados dois modelos principais: (i) uma CNN simples, construída do zero, (ii) CNN + Autoencoder e (iii) uma CNN mais profunda baseada em uma arquitetura semelhante à ResNet, ambas utilizando PyTorch.

Após o pré-processamento, as imagens foram organizadas em tensores no formato (N, C, H, W) e divididas em três subconjuntos:

- Treinamento: 4.854 imagens
- Validação: 675 imagens
- Teste: 1.214 imagens

As etiquetas foram convertidas para inteiros consistentes com o mapeamento da base.

Antes do treinamento, os tensores foram normalizados canal a canal, utilizando média e desvio-padrão calculados diretamente no conjunto de treinamento. Isso garante maior estabilidade na otimização.

3.4.1. CNN Simples

O primeiro classificador consiste em uma arquitetura com duas camadas convolucionais seguidas de Batch Normalization, função de ativação ReLU e operações de MaxPooling. Após as convoluções, os mapas de ativação são achatados e conectados a uma camada totalmente conectada com 9 saídas (uma para cada classe).

Detalhes principais da arquitetura:

- Conv2D(3→4 filtros) + BatchNorm + ReLU + MaxPool
- Conv2D(4→4 filtros) + BatchNorm + ReLU + MaxPool
- Flatten
- Linear(3136 → 9)

O modelo foi treinado por 80 épocas usando:

- Adam (lr=0.0001)
- CrossEntropyLoss com pesos de classe, calculados a partir do desbalanceamento da base
- Treinamento realizado em GPU (CUDA)

O desempenho final obtido no conjunto de teste foi:

- Acurácia total: ~66%
- Classes com melhor desempenho: *Normal*, *Alterações de Mediastino*, *Alterações de Tórax*
- Classes mais desafiadoras: *Obstrutivas* e *Maior Densidade*, devido à sobreposição visual entre subtipos.

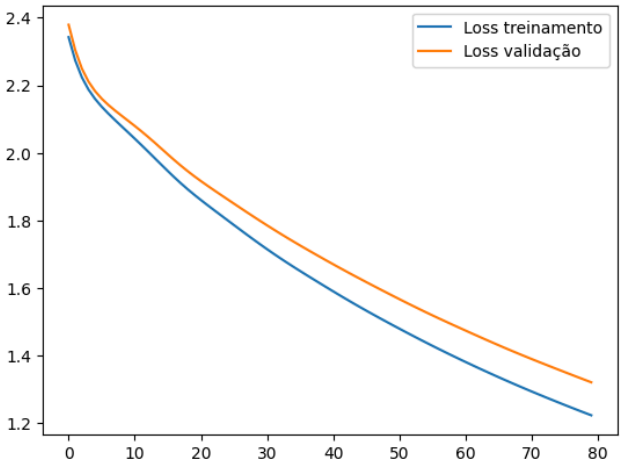

Figura 5

A matriz de confusão indicou que boa parte dos erros ocorreu entre categorias com características radiológicas semelhantes, o que motivou a adoção de um modelo mais complexo.

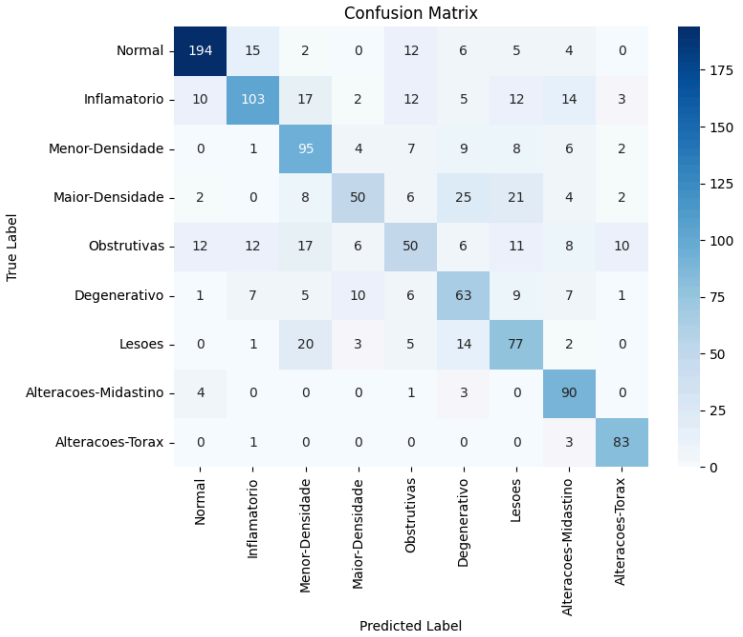

Figura 6

3.4.2 Autoencoder + CNN Simples

Com o objetivo de melhorar a CNN Simples, foi aplicada a técnica de Autoencoder + CNN Simples. Basicamente, a ideia é a seguinte: utilização de um Autoencoder convolucional para aprender uma representação compacta das imagens antes da etapa de classificação.

A ideia principal é que o Autoencoder reduza ruídos, intensifique padrões estruturais das radiografias e forneça um espaço mais discriminativo, facilitando o trabalho da CNN de classificação. Assim como no modelo anterior, a arquitetura final do classificador consiste em uma CNN simples, porém agora aplicada sobre imagens reconstruídas, mais homogêneas e menos ruidosas.

O Autoencoder foi treinado de forma não supervisionada e dividido em duas partes: um encoder convolucional, responsável por reduzir progressivamente a dimensionalidade da entrada, e um decoder simétrico, que reconstrói a imagem original. Após o treinamento, somente o encoder é utilizado como pré-processamento, alimentando a CNN classificadora.

Detalhes principais da arquitetura:

- Encoder:
    - Conv2D → BatchNorm → ReLU;
    - Conv2D → BatchNorm → ReLU;
    - MaxPool;
    - Conv2D → BatchNorm → ReLU;
    - MaxPool;
- Decoder:
    - ConvTranspose2D → BatchNorm → ReLU;
    - Upsampling;
    - ConvTranspose2D → BatchNorm → ReLU;
    - Upsampling;
    - ConvTranspose2D (reconstrução final);
- Classificador (CNN após Autoencoder):
    - Conv2D(3→4 filtros) + BatchNorm + ReLU + MaxPool;
    - Conv2D(3→4 filtros) + BatchNorm + ReLU + MaxPool;
    - Flatten;
    - Linear → 9 classes;

O modelo foi treinado por 80 épocas usando:

- Adam (lr=0.0001);
- CrossEntropyLoss com pesos de classe;
- Treinamento realizado em GPU (CUDA);
- Reconstruções do Autoencoder como entrada para a CNN.

Quanto ao desempenho e comportamento do treinamento, a curva de loss apresentou queda contínua tanto em treinamento quanto em validação, indicando bom ajuste sem sinais severos de sobreajuste. A perda final ficou próxima de 0.95 (treinamento) e ~1.0 (validação) após as 80 épocas.

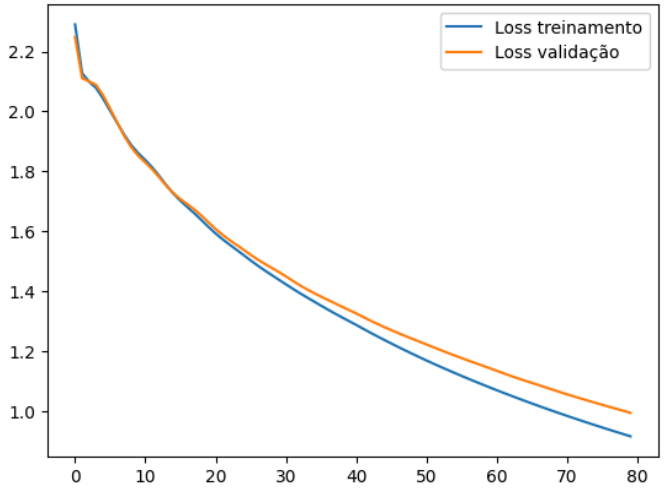

Figura 7

O desempenho final no conjunto de teste foi:

- Acurácia total: ~77%, superior à obtida pela CNN Simples isolada;
- Melhores desempenhos: *Alterações de Mediastino, Alterações de Tórax, Normal* e *Maior Densidade;* Mais desafiadoras: *Obstrutivas, Menor Densidade* e *Degenerativo*;

A matriz de confusão evidencia que o modelo reduziu erros entre grupos anteriormente muito sobrepostos, especialmente em *Alterações de Tórax, Lesões* e *Maior Densidade*, que obtiveram ganhos claros em recall. Contudo, ainda há confusão entre *Obstrutivas, Menor Densidade* e *Degenerativo*, cujas características radiológicas podem ser sutis ou compartilhadas entre subtipos.

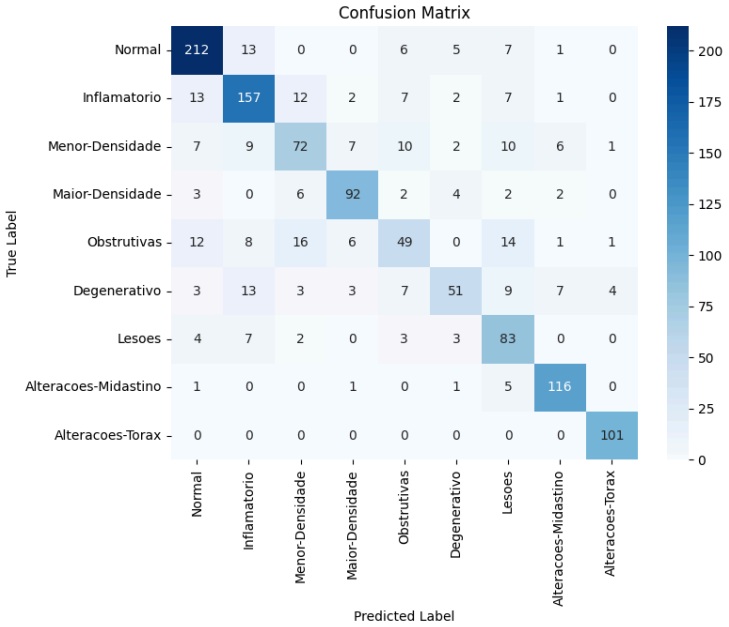

Figura 8

As amostras qualitativas mostram que o modelo acerta consistentemente imagens com padrões radiológicos mais proeminentes (Alterações de Mediastino, Lesões e Normal). Nas imagens incorretas, observa-se que o erro geralmente ocorre entre classes visualmente próximas.

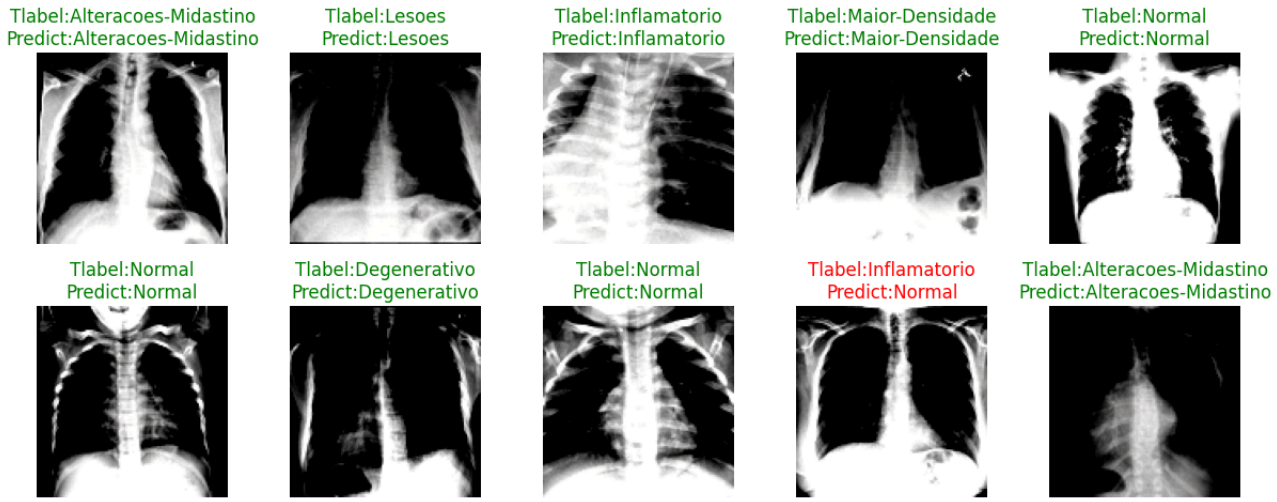

Figura 9

A reconstrução do Autoencoder tende a suavizar ruídos, mas em alguns casos também reduz detalhes sutis, o que pode levar à classificação equivocada, especialmente em patologias com fronteiras pouco definidas.

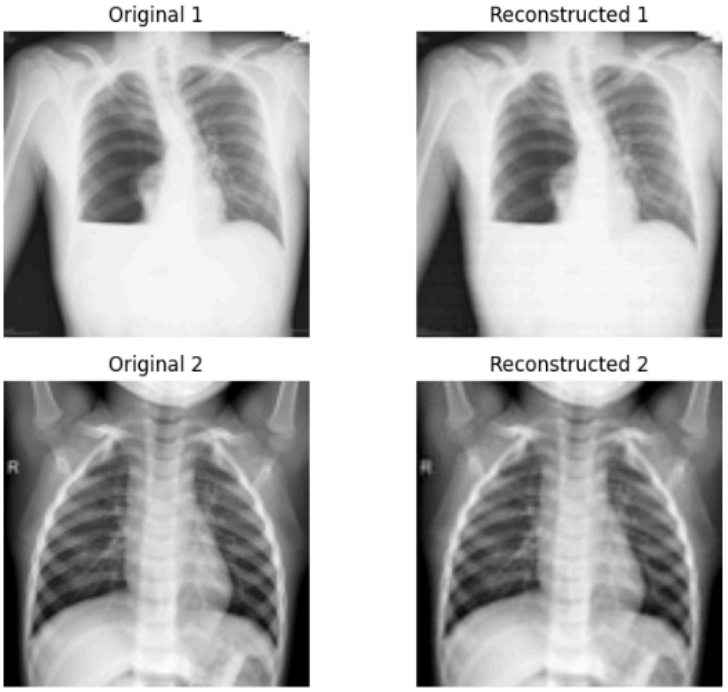

Figura 10a

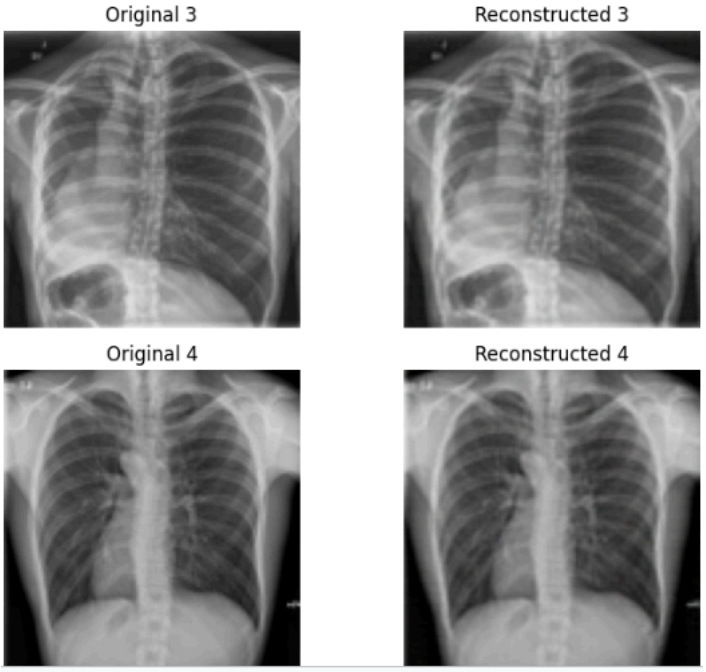

Figura 10b

3.4.3 Modelo Baseado em ResNet

Visando maior capacidade de extração de padrões, foi implementada uma rede inspirada na ResNet, usando blocos residuais do tipo *BasicBlock*. A arquitetura inclui:

- Convolução inicial (7×7) + MaxPool
- Três estágios residuais com 64, 128 e 512 filtros
- Global Average Pooling
- Camada final Linear(512 → 9)

O modelo foi treinado com:

- Adam (lr=0.001)
- CrossEntropyLoss com pesos de classe
- Mini-batches de 32 imagens via DataLoader
- ReduceLROnPlateau como scheduler, diminuindo a taxa de aprendizado quando a perda de validação estagnava.

Esse modelo apresentou desempenho superior aos dois anteriores, alcançando:

- Acurácia no teste: ~97%

- Melhora expressiva em Menor Densidade, Degenerativo e Lesões
- Menor overfitting graças a batch norm + convoluções profundas + skip connections

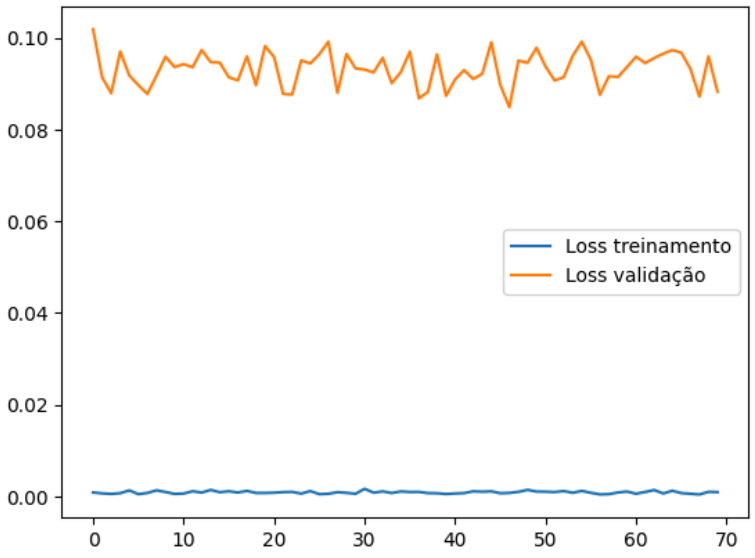

Figura 11

A matriz de confusão confirmou maior separabilidade entre categorias, indicando que o modelo consegue extrair representações mais discriminativas.

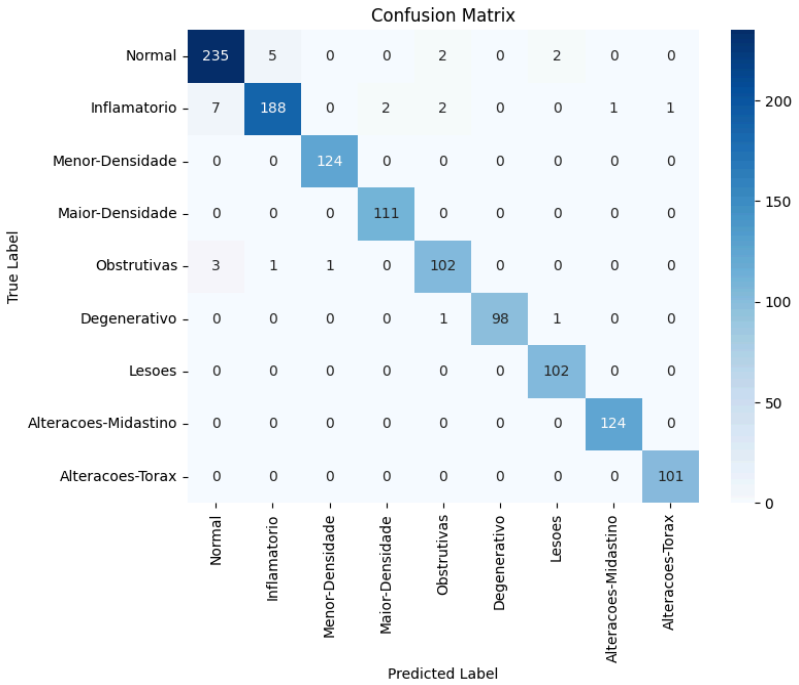

Figura 12

4. Visualização e Comparação dos Resultados

Observamos uma evolução clara de desempenho: enquanto o Autoencoder melhorou a base da CNN Simples **(subindo de ~69% para ~76%**), a ResNet foi definitiva, superando ambas com uma margem de **mais de 20 pontos** percentuais.

Modelo	Acurácia	Precisão (W)	Recall (W)	F1-Score (W)
Simple CNN	0.6936	0.6935	0.6936	0.6889
Autoencoder + CNN	0.7685	0.7658	0.7685	0.7628
ResNet	**0.9761**	**0.9761**	**0.9761**	**0.9760**

Figura 13

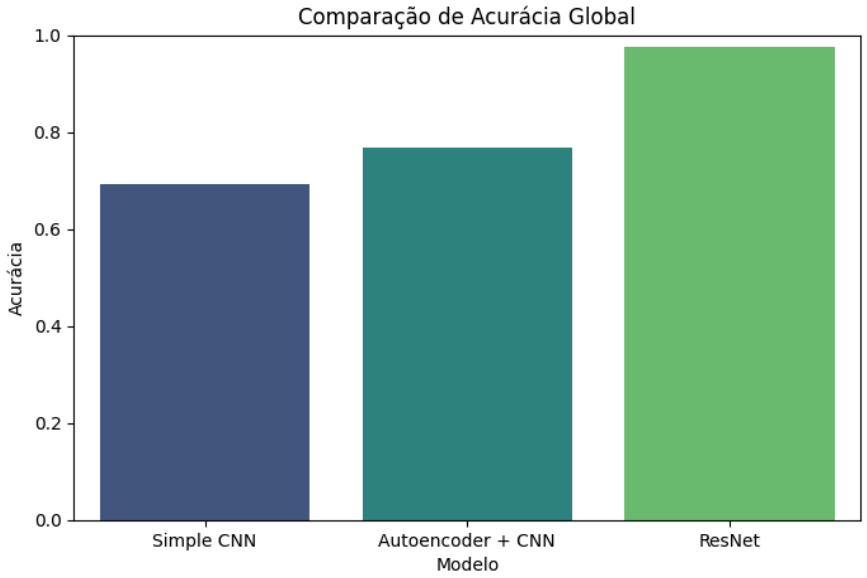

Figura 14

Análise de Confiabilidade: Enquanto a Simple CNN e o modelo híbrido (Autoencoder) apresentam índices de concordância moderados a substanciais (0.65 e 0.73, respectivamente), a ResNet atinge um patamar de excelência. Com Cohen's Kappa e MCC superiores a 0.97, a ResNet demonstra uma capacidade de generalização quase perfeita, indicando que o modelo não sofre de vieses significativos e mantém alta performance mesmo em classes desbalanceadas.

Modelo	Cohen's Kappa	MCC
Simple CNN	0.6508	0.6520
Autoencoder + CNN	0.7349	0.7358
ResNet	0.9727	0.9727

Figura 15

Observamos também que o modelo ResNet apresenta os maiores F1-scores em praticamente todas as classes, frequentemente próximos de 0.95–1.00, indicando alta precisão e recall simultaneamente. O modelo Autoencoder + CNN, por sua vez, supera consistentemente a CNN Simples em quase todas as categorias, mantendo ganhos especialmente visíveis em classes como Inflamatório, Maior Densidade, Lesões e Alterações de Mediastino.

Já a CNN Simples apresenta desempenho inferior, principalmente em classes mais difíceis como Obstrutivas, Degenerativo e Menor Densidade, refletindo sua limitação na extração de padrões mais sutis nas imagens.

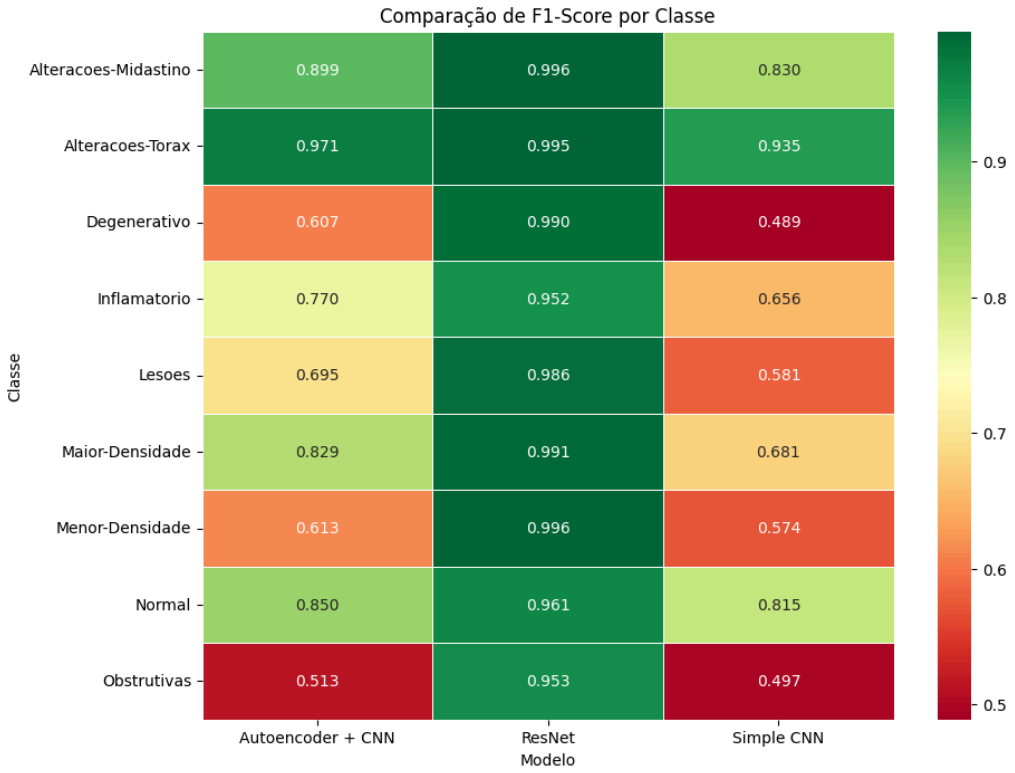

Figura 17

Importante ressaltar:

- **Uniformidade:** Enquanto os modelos Simple CNN e Autoencoder oscilam drasticamente dependendo da patologia (note as áreas em vermelho/laranja), a ResNet é o único modelo que não apresenta "pontos cegos" (nenhuma classe abaixo de 0.90).
- **Performance Máxima**: Atingiu F1-Score perfeito de 1.000 em 'Alterações no Tórax'.
- **Resiliência:** Mesmo nas classes onde a Simple CNN colapsa (ex: Obstrutivas e Degenerativo), a ResNet mantém a excelência.

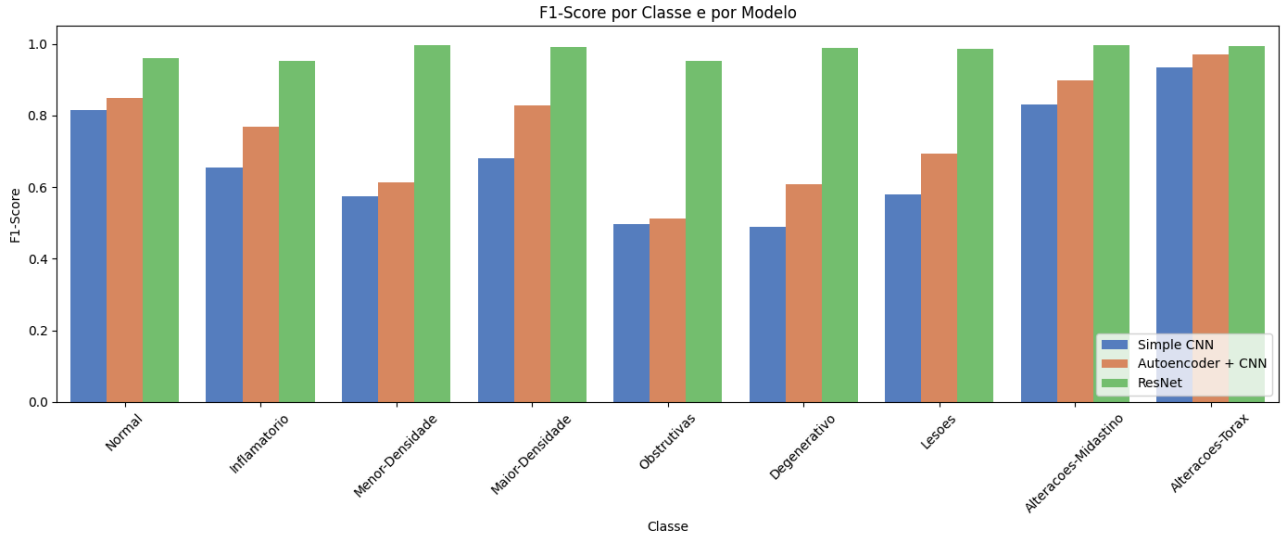

Figura 16

É notável a disparidade em classes complexas. Em 'Obstrutivas' e 'Degenerativo', as barras azul e laranja mostram que os modelos simples colapsaram (ficaram próximos de 0.5). A ResNet, por outro lado, manteve a estabilidade, provando que sua profundidade permite discriminar características que passariam despercebidas por redes mais rasas.

5. Conclusão

O presente trabalho teve como objetivo avaliar e comparar a eficácia de diferentes arquiteturas de Redes Neurais Convolucionais (CNNs) na detecção automática de pneumonia em imagens de Raio-X de tórax. A análise comparativa entre uma CNN Simples, uma abordagem híbrida com Autoencoder e a arquitetura ResNet permitiu identificar diferenças significativas em capacidade de aprendizado e generalização.

Os resultados obtidos demonstraram uma clara evolução de desempenho conforme a complexidade da arquitetura aumentou. A Simple CNN e o modelo Autoencoder + CNN apresentaram resultados moderados, com acurácias de aproximadamente 69% e 76%, respectivamente. Embora o uso do Autoencoder tenha melhorado a extração de características latentes em relação à rede simples, ambas as abordagens mostraram limitações na consistência das classificações, evidenciadas por índices Kappa e MCC inferiores a 0.75.

Em contraste, a arquitetura ResNet consolidou-se como a solução mais robusta para o problema proposto. Alcançando uma acurácia global de 97,61% e métricas de confiabilidade (Cohen's Kappa e MCC) superiores a 0.97, o modelo provou ser altamente eficaz. Esse desempenho superior justifica-se pela utilização de conexões residuais, que permitiram o treinamento de uma rede profunda capaz de capturar nuances visuais complexas da patologia sem sofrer com a degradação do gradiente.

Conclui-se, portanto, que a arquitetura ResNet não apenas superou os modelos de base, mas atingiu níveis de excelência estatística que sugerem sua viabilidade como uma ferramenta de auxílio ao diagnóstico médico (CAD). O modelo oferece um equilíbrio ideal entre precisão e revocação (Recall), garantindo segurança na detecção tanto de casos positivos quanto negativos.

6. Referências

Aula sobre CNN

https://www.kaggle.com/datasets/tolgadincer/labeled-chest-xray-images

Mustafa Qafesha R, Diaa Hindawi M, Elbadry M, Sharabati I, Mashaly D, Dervis M, Najah Q, Kashbour M. Chest radiography in interstitial lung disease: Accuracy and radiological features from a systematic review and meta-analysis. Sarcoidosis Vasc Diffuse Lung Dis. 2025 Jun 25;42(2):16108. doi: 10.36141/svdld.v42i2.16108. PMID: 40699492; PMCID: PMC12242875.

https://jornal.usp.br/radio-usp/uso-da-ia-no-instituto-de-radiologia-da-usp-permitira-laudos-mais-ageis/

https://g1.globo.com/google/amp/pr/parana/especial-publicitario/metodo-clinica-futurista/noticia/2025/05/09/o-impacto-da-inteligencia-artificial-na-radiologia-diagnostico-preciso-fluxo-agil-e-tratamento-personalizado.ghtml